🛠️ Technology-Specific Practice Collection

This folder contains hands-on practice with specific technologies commonly used in distributed systems and backend development.

🎯 Purpose

Master integration with popular backend technologies through practical examples:

- Redis (In-memory data store & caching)
- Kafka (Distributed streaming platform)
- Real-world usage patterns
- Interview preparation for technology-specific questions

📁 Files Overview

1. **RedisPractice.java**

Technology: Redis - In-Memory Data Store

Topics Covered:

- Jedis client usage
- String operations (GET, SET, TTL)
- Data structures (Lists, Sets, Sorted Sets, Hashes)
- Pub/Sub messaging
- Connection pooling
- Expiration and TTL
- Caching patterns

Use Cases:

- Session storage
- Caching layer
- Real-time leaderboards
- Rate limiting
- Distributed locks

Interview Topics:

- Redis data structures and time complexity
- Cache eviction policies (LRU, LFU)
- Redis persistence (RDB, AOF)
- Redis Cluster vs Sentinel
- When to use Redis vs Memcached

---

2. **RedisInterviewPractice.java**

Technology: Redis - Interview-Focused Problems

Topics Covered:

- Real-world interview scenarios
- Advanced Redis patterns
- Performance optimization
- Cache strategies (write-through, write-back, write-around)
- Distributed caching challenges

Common Interview Questions:

- "Design a distributed cache"
- "Implement rate limiting with Redis"
- "Design a session store"
- "How would you handle cache invalidation?"
- "Explain Redis persistence options"

---

3. **KafkaPractice.java**

Technology: Apache Kafka - Distributed Streaming

Topics Covered:

- Producer API (send messages)
- Consumer API (poll, consume, commit)
- Topics, partitions, and offsets
- Consumer groups
- Serialization/Deserialization
- Error handling and retries
- Idempotent producers

Use Cases:

- Event streaming
- Log aggregation
- Real-time analytics
- Microservices communication
- CDC (Change Data Capture)

Interview Topics:

- Kafka architecture (brokers, topics, partitions)
- Consumer groups and rebalancing
- Message ordering guarantees
- At-least-once vs exactly-once semantics
- Kafka vs RabbitMQ vs SQS

- Partition key selection

🚀 Setup Instructions

Redis Setup:

```
# Start Redis server
redis-server

# Or via Docker
docker run -d -p 6379:6379 redis:latest

# Test connection
redis-cli ping  # Should return PONG
```

Kafka Setup:

```
# Using provided script
./run-kafka.sh

# Or manually
# 1. Start Zookeeper
# 2. Start Kafka broker
# 3. Create topics
```

Running Practice Files:

```
cd Technology_Practice

# Compile with dependencies (classpath includes jedis & kafka jars)
javac -cp "../jedis-5.1.0.jar:../commons-pool2-2.12.0.jar:."
RedisPractice.java
javac -cp "../kafka-clients-3.6.1.jar:../slf4j-api-2.0.9.jar:."
KafkaPractice.java

# Run
java -cp "../jedis-5.1.0.jar:../commons-pool2-2.12.0.jar:."
RedisPractice
java -cp "../kafka-clients-3.6.1.jar:../slf4j-api-2.0.9.jar:../slf4j-
simple-2.0.9.jar:." KafkaPractice
```

📊 Practice Tracker

Technology	File	Size	Status	Last Practiced	Dependencies
Redis	RedisPractice.java	-	✅	-	jedis-5.1.0.jar
Redis Interview	RedisInterviewPractice.java	-	✅	-	jedis-5.1.0.jar
Kafka	KafkaPractice.java	-	✅	-	kafka-clients-3.6.1.jar

🎯 Interview Focus Areas

Redis (High Frequency in Interviews)

Core Concepts:

- ✅ Data structures: String, List, Set, Sorted Set, Hash
- ✅ Time complexity: O(1) for most operations
- ✅ Persistence: RDB snapshots vs AOF logs
- ✅ Replication: Master-Slave architecture
- ✅ Eviction policies: LRU, LFU, random, TTL

Common Questions:

1. "How would you implement a cache with Redis?"
2. "Design a rate limiter using Redis"
3. "How to handle cache stampede?"
4. "Explain Redis Cluster sharding"
5. "Redis vs Memcached - when to use which?"

Design Patterns:

- Cache-Aside (lazy loading)
- Write-Through (sync writes)
- Write-Behind (async writes)
- Refresh-Ahead (predictive loading)

Kafka (Medium-High Frequency)

Core Concepts:

- ✅ Topics, Partitions, Offsets
- ✅ Producer: send(), async/sync, idempotence
- ✅ Consumer: poll(), commit strategies
- ✅ Consumer Groups: load balancing, failover
- ✅ Replication: Leader-Follower, ISR

Common Questions:

1. "Design an event-driven system with Kafka"
2. "How does Kafka achieve high throughput?"
3. "Explain consumer group rebalancing"
4. "How to ensure exactly-once processing?"
5. "Kafka vs RabbitMQ - trade-offs?"

Key Scenarios:

- Real-time analytics pipeline
- Microservices event bus
- Log aggregation
- CDC (Change Data Capture)
- Stream processing

💡 Common Patterns

Pattern 1: Redis Caching

```
// Try cache first, then database
String value = redis.get(key);
if (value == null) {
    value = database.query(key);
    redis.setex(key, TTL, value);
}
return value;
```

Pattern 2: Redis Rate Limiting

```
// Token bucket using Redis
String key = "rate_limit:" + userId;
long count = redis.incr(key);
if (count == 1) {
    redis.expire(key, windowSeconds);
}
return count <= maxRequests;
```

Pattern 3: Kafka Consumer with Manual Commit

```
while (true) {
    ConsumerRecords<K,V> records =
consumer.poll(Duration.ofMillis(100));
    for (ConsumerRecord<K,V> record : records) {
        process(record);
    }
```

```
    consumer.commitSync(); // Manual commit after processing
}
```

🔥 Quick Reference

Redis Data Structures & Use Cases:

Data Type	Use Case	Example
String	Cache, Counter, Session	`SET user:1001 "data"`
List	Queue, Timeline, Recent items	`LPUSH queue task1`
Set	Tags, Unique visitors	`SADD tags:post1 "java"`
Sorted Set	Leaderboard, Priority queue	`ZADD leaderboard 100 user1`
Hash	Object storage	`HSET user:1001 name "John"`

Kafka Components:

- **Producer**: Sends messages to topics
- **Consumer**: Reads messages from topics
- **Broker**: Kafka server storing messages
- **Topic**: Category/feed of messages
- **Partition**: Ordered log within topic
- **Consumer Group**: Load balancing consumers
- **Offset**: Position in partition

📖 Technology Comparison

Caching: Redis vs Memcached

Feature	Redis	Memcached
Data Structures	✅ Rich (List, Set, Hash)	❌ Only String
Persistence	✅ Yes (RDB, AOF)	❌ No
Replication	✅ Yes	❌ No
Transactions	✅ Yes	❌ No
Pub/Sub	✅ Yes	❌ No
Speed	⚡ Fast	⚡⚡ Slightly faster

Messaging: Kafka vs RabbitMQ

Feature	Kafka	RabbitMQ
Throughput	⚡⚡⚡ Very High	⚡⚡ High
Message Retention	✅ Yes (configurable)	❌ Consumed = deleted
Message Ordering	✅ Per partition	✅ Per queue
Use Case	Event streaming, logs	Task queues, RPC
Complexity	Higher (distributed)	Lower (simpler setup)

🏆 Interview Preparation

Redis Interview Checklist:

- ☐ Understand all 5 data structures and time complexity
- ☐ Know cache eviction policies
- ☐ Explain persistence options (RDB vs AOF)
- ☐ Describe Redis Cluster sharding
- ☐ Discuss common caching patterns
- ☐ Handle cache stampede scenario

Kafka Interview Checklist:

- ☐ Explain Kafka architecture (brokers, topics, partitions)
- ☐ Understand consumer groups and rebalancing
- ☐ Know delivery semantics (at-least-once, exactly-once)
- ☐ Discuss partition key selection strategy
- ☐ Explain offset management
- ☐ Compare Kafka with alternatives (RabbitMQ, SQS)

📚 Additional Resources

Redis:

- Official Docs: redis.io/documentation
- Redis University: university.redis.com (free courses)
- Book: "Redis in Action"

Kafka:

- Official Docs: kafka.apache.org/documentation
- Confluent Tutorials: developer.confluent.io
- Book: "Kafka: The Definitive Guide"

🎓 Learning Path

Week 1-2: Redis Fundamentals

- Data structures practice
- Caching patterns
- Common interview problems

Week 3-4: Kafka Fundamentals

- Producer/Consumer APIs
- Partition and offset management
- Consumer groups

Week 5: Integration & System Design

- Design systems using Redis + Kafka
- Real-world architecture discussions
- Scalability considerations

Master these technologies and become a distributed systems expert! 🚀

Last Updated: March 16, 2026